Exercise 1: Kubernetes — Hello Pod

DevOps Lab | Practical steps and last-minute revision notes

Completion status: VERIFIED — Nginx welcome page successfully opened in the browser.

Objective

Deploy an Nginx Pod in a local Kubernetes cluster, expose it using a NodePort Service, and verify it in a browser.

Environment used

• Windows 11 PowerShell

• Docker Desktop using the desktop-linux context and WSL2

• Minikube v1.38.1 with Kubernetes v1.35.1

• kubectl client v1.30.2 (older than the cluster version; use Minikube's bundled kubectl if compatibility problems occur)

Procedure and commands

1. Check installed tools

```
docker --version
minikube version
kubectl version --client
minikube status
```

If Docker Desktop is stopped, open it and wait until its Linux engine is running. Verify Docker connectivity with:

```
docker info
docker ps
```

2. Start Minikube

```
minikube start --driver=docker
```

Verify that the control plane and node are healthy:

```
minikube status
kubectl get nodes
```

Expected: the Minikube node reports STATUS Ready. In this run, Kubernetes v1.35.1 was running successfully.

3. Create the Nginx Pod

```
kubectl run hello-k8s --image=nginx --port=80
```

Meaning: kubectl run creates a Pod named hello-k8s, uses the nginx image, and declares container port 80.

4. Verify Pod readiness

```
kubectl get pods
```

Wait if STATUS is ContainerCreating. Continue once READY is 1/1 and STATUS is Running.

5. Expose the Pod with a NodePort Service

```
kubectl expose pod hello-k8s --type=NodePort --port=80
```

A Service gives clients a stable way to reach the Pod. NodePort exposes the Service on a port on the node.

6. Verify the Service

```
kubectl get services
```

Observed result: hello-k8s was a NodePort Service with port mapping 80:31768/TCP. Kubernetes assigned ClusterIP 10.111.109.155 in this run. These addresses/ports may differ on another run.

7. Open Nginx

```
minikube service hello-k8s
```

Minikube opened the Nginx welcome page. On Windows with the Docker driver, Minikube may start a local tunnel and display a URL such as http://127.0.0.1:57355. Keep that PowerShell terminal open while using the tunnel.

Architecture / flow

Browser → NodePort Service → Nginx Pod → Nginx container listening on port 80

Pod: runs the application container. Service: stable networking endpoint for accessing the Pod. NodePort: exposes a Service through a port on the node. Minikube: local Kubernetes cluster. kubectl: CLI used to manage Kubernetes resources.

Verification checklist

[x] Docker engine responded to docker info.

[x] Minikube status showed host, kubelet and apiserver Running.

[x] kubectl get nodes showed minikube as Ready.

[x] Pod hello-k8s reached 1/1 Running.

[x] NodePort Service hello-k8s was created.

[x] Browser displayed “Welcome to nginx!” at the local forwarded URL.

Troubleshooting

Docker connection error mentioning dockerDesktopLinuxEngine

Start Docker Desktop and wait for the Linux engine. Check docker info and docker ps, then retry minikube start --driver=docker.

Pod remains ContainerCreating

Wait and run kubectl get pods again. For more detail, use kubectl describe pod hello-k8s.

ImagePullBackOff

Inspect the Pod events with kubectl describe pod hello-k8s. Check network access and image-pull errors.

Browser URL works only while the terminal is open

Expected with the Minikube Docker driver on Windows when a tunnel is used. Keep the terminal running during the test.

kubectl client/cluster version mismatch

This environment has kubectl v1.30.2 and Kubernetes v1.35.1. If a command behaves unexpectedly, try Minikube's bundled kubectl: minikube kubectl -- get pods. For example: minikube kubectl -- get nodes.

Cleanup (optional)

Use cleanup only after you no longer need the exercise resources. These commands delete the Pod and Service, not the Minikube cluster:

```
kubectl delete service hello-k8s
kubectl delete pod hello-k8s
```

To stop the cluster without deleting it: minikube stop. Do not delete resources until you have finished any required screenshots or demonstrations.

Exam quick revision

What is a Pod? The smallest deployable unit in Kubernetes; it can contain one or more closely related containers.

Why use a Service? It provides a stable endpoint to reach a changing set of Pods.

What does NodePort do? Exposes a Service on a port on each node, making it reachable from outside the cluster when network routing permits.

What does --image=nginx specify? The container image Kubernetes should run.

What does --port=80 mean here? It declares the container port; it does not, by itself, expose the Pod outside the cluster.

Why did the browser use 127.0.0.1:57355? Minikube created a local tunnel for the Service on Windows with the Docker driver.

How do you check Pod status? kubectl get pods; use kubectl describe pod hello-k8s for detailed events.

How do you check cluster nodes? kubectl get nodes.

Successful run summary

```
Pod:       hello-k8s
Image:     nginx
Pod state: 1/1 Running
Service:   hello-k8s (NodePort)
Port:      80:31768/TCP (observed in this run)
Browser:   Nginx Welcome page displayed
Result:    Exercise 1 completed successfully
```